

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 1 实验名称 算术逻辑单元 ALU 与寄存器堆设计实验

注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。

注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。

注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与仿真波形的截图及说明

本次实验完成了 32 位 MIPS 架构算术逻辑单元(ALU)与 32×32 位通用寄存器堆的 RTL 设计、功能仿真与验证, 以下分模块进行详细说明。

1. 寄存器堆设计

- (a) 模块功能与逻辑结构本模块实现符合 MIPS 架构规范的 32 个 32 位通用寄存器堆, 核心特性包括: 双端口异步组合逻辑读、单端口同步时序逻辑写、\$zero 寄存器(地址 0)恒为 0 且禁止写入。模块整体逻辑结构如 Fig. 1 所示。

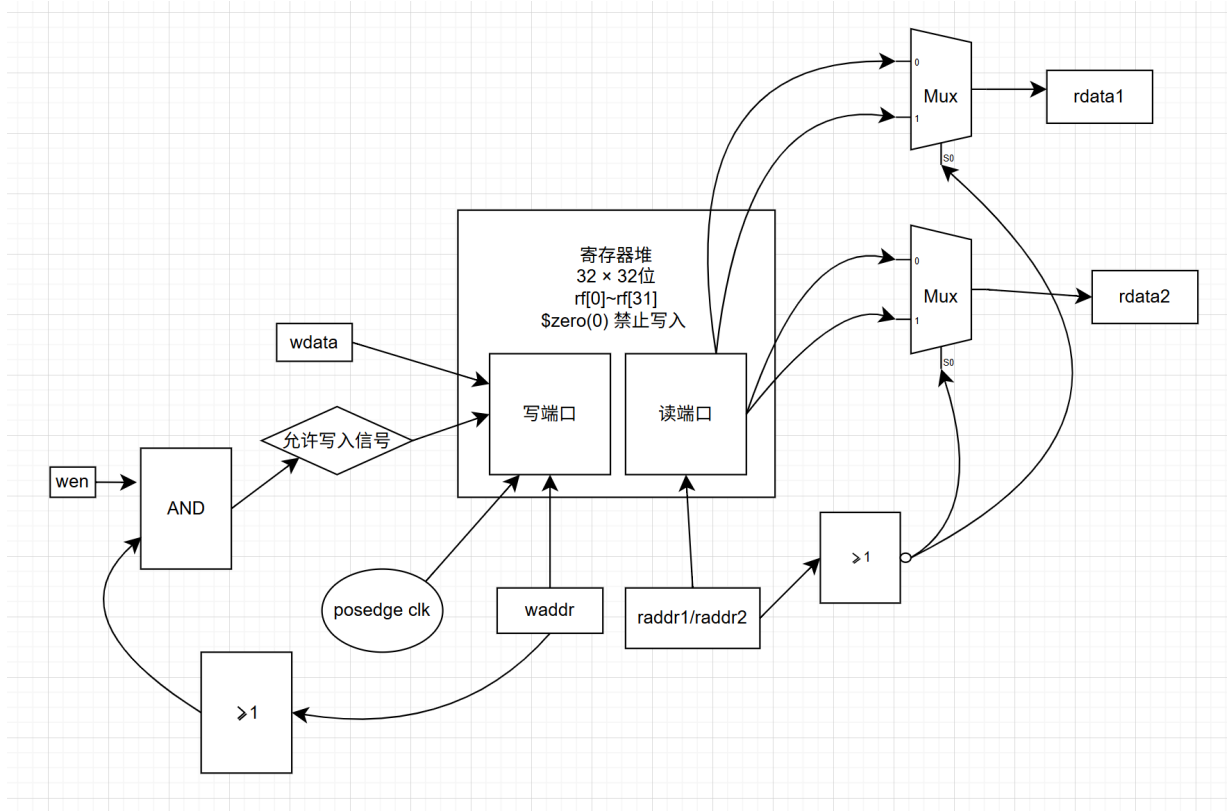


图 1: 寄存器堆逻辑结构框图

模块核心逻辑分为三部分:

- 读端口逻辑: 通过组合逻辑实现地址译码与数据输出, 当读地址为 0 时强制输出 32 位全 0, 否则输出对应地址寄存器的值;
- 写端口逻辑: 时钟上升沿触发, 仅当写使能有效且写地址不为 0 时, 将写入数据更新到对应地址的寄存器中;
- 寄存器存储体: 定义 32 个 32 位寄存器数组, 作为数据存储载体。

(b) RTL 核心代码模块完整 Verilog HDL 代码如下, 通过宏定义配置位宽, 具备良好的可扩展性。

```

1  `timescale 10 ns / 1 ns
2  `define DATA_WIDTH 32
3  `define ADDR_WIDTH 5
4  module reg_file(
5      input          clk,
6      input [`ADDR_WIDTH - 1:0] waddr,
7      input [`ADDR_WIDTH - 1:0] raddr1,
8      input [`ADDR_WIDTH - 1:0] raddr2,
9      input          wen,
10     input [`DATA_WIDTH - 1:0] wdata,
11     output [`DATA_WIDTH - 1:0] rdata1,
12     output [`DATA_WIDTH - 1:0] rdata2
13 );
14 // 定义32个32-bit寄存器数组
15 reg [`DATA_WIDTH - 1:0] rf [0:(1 << `ADDR_WIDTH) - 1];
16 // 读端口1: 逻辑运算实现$zero恒0 (组合逻辑)

```

```

17 assign rdata1 = ({`DATA_WIDTH{~|raddr1|}} & rf[raddr1]) | ({`DATA_WIDTH{~|raddr1|}} &
    `DATA_WIDTH'd0);
18 // 读端口2: 逻辑运算实现$zero恒0 (组合逻辑)
19 assign rdata2 = ({`DATA_WIDTH{~|raddr2|}} & rf[raddr2]) | ({`DATA_WIDTH{~|raddr2|}} &
    `DATA_WIDTH'd0);
20 // 写端口: 同步写 + 禁止写$zero (时序逻辑)
21 always @(posedge clk) begin
22     if (wen && (waddr != `ADDR_WIDTH'b0)) begin
23         rf[waddr] <= wdata;
24     end
25 end
26
27 endmodule

```

(c) 仿真结果说明本模块通过功能仿真完成全功能验证, 覆盖 \$zero 寄存器恒零特性、双端口异步读正确性、同步写时序合规性、地址 0 写保护功能, 所有仿真结果均符合设计预期。

2. ALU 设计

(a) 模块功能与逻辑结构本模块实现 32 位 MIPS 架构算术逻辑单元, 支持 AND、OR、ADD、SUB、SLT 共 5 种运算, 通过 3 位 ALUop 编码选择运算类型, 同时输出溢出标志 Overflow、进位/借位标志 CarryOut、零标志 Zero 三个状态标志位。模块整体逻辑结构如 Fig. 2 所示。

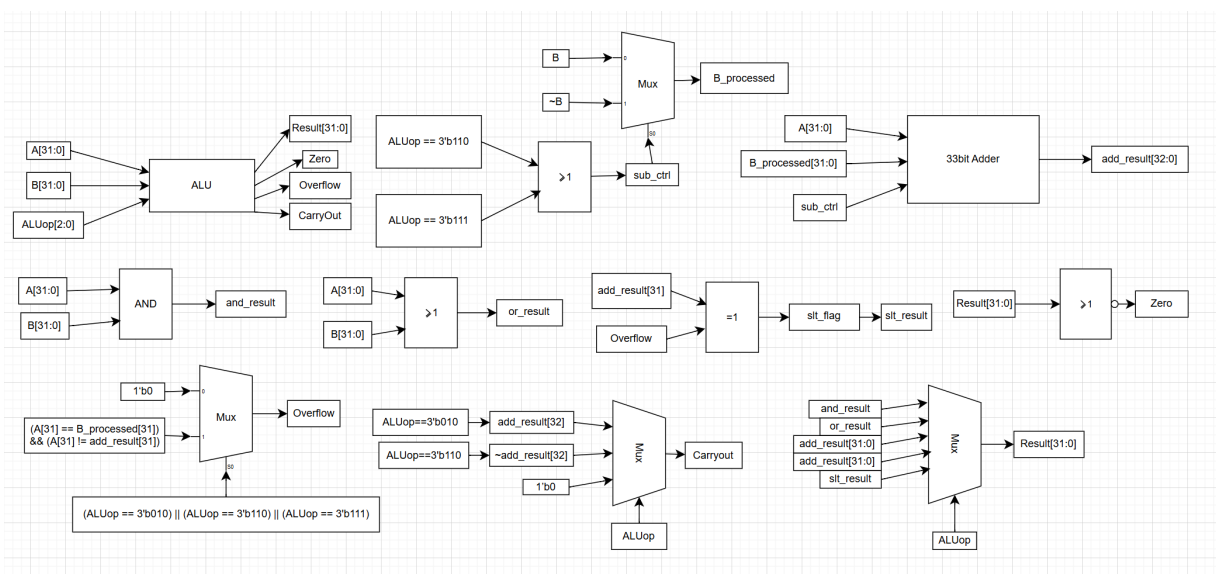


图 2: ALU 逻辑结构框图

模块核心逻辑分为三部分:

- 运算单元逻辑: 复用同一套 33 位加法器实现 ADD、SUB、SLT 运算, 通过 sub_ctrl 控制信号实现补码减法的取反加一操作; 并行实现 AND、OR 按位逻辑运算;
- 结果选择逻辑: 通过多路选择器 MUX, 根据 ALUop 编码选择对应运算结果作为最终输出;
- 标志位生成逻辑: 组合逻辑实现三个状态标志位计算, Zero 标志对所有运算有效, Overflow 和 CarryOut 仅对算术运算有效。

(b) RTL 核心代码模块完整 Verilog HDL 代码如下, 严格遵循 MIPS ALUop 编码规范, 所有逻辑均为可综合设计。

```

1  `timescale 10 ns / 1 ns
2  `define DATA_WIDTH 32
3  module alu(
4      input [`DATA_WIDTH - 1:0] A,
5      input [`DATA_WIDTH - 1:0] B,
6      input [          2:0] ALUop,
7      output                Overflow,
8      output                CarryOut,
9      output                Zero,
10     output [`DATA_WIDTH - 1:0] Result
11 );
12 // ===== 第一步: 定义中间信号 (纯wire, 组合逻辑) =====
13 // 1. 减法控制信号: ALUop=110(SUB)/111(SLT)时, 需要执行减法 (A-B)
14 wire sub_ctrl = (ALUop == 3'b110) || (ALUop == 3'b111);
15 // 2. B操作数处理: 减法时取反B (补码减法: A-B = A + (~B) + 1)
16 wire [`DATA_WIDTH - 1:0] B_processed = sub_ctrl ? ~B : B;
17 // 3. 加法器核心: 复用同一套加法器实现ADD/SUB/SLT
18 //   add_result[32]: 最高位为进位/借位 (CarryOut), 低32位为运算结果
19 wire [32:0] add_result = A + B_processed + sub_ctrl; // +sub_ctrl实现补码减法的+1
20 // 4. 各运算的中间结果 (并行计算, 组合逻辑)
21 wire [`DATA_WIDTH - 1:0] and_result = A & B; // AND运算结果
22 wire [`DATA_WIDTH - 1:0] or_result = A | B; // OR运算结果
23 // 5. SLT (有符号比较) 结果: A < B 时最低位为1, 其余为0
24 //   判定逻辑: 减法结果的符号位异或溢出标志, 为1则A < B
25 wire slt_flag = add_result[31] ^ Overflow;
26 wire [`DATA_WIDTH - 1:0] slt_result;
27 assign slt_result[0] = slt_flag; // 最低位赋值为比较结果
28 assign slt_result[31:1] = 31'd0; // 高31位直接赋0
29 // ===== 第二步: 运算结果选择 (MUX, 组合逻辑) =====
30 // 根据ALUop选择最终输出的Result, 严格匹配MIPS ALUop编码
31 assign Result = (ALUop == 3'b000) ? and_result : // AND运算
32                 (ALUop == 3'b001) ? or_result : // OR运算
33                 (ALUop == 3'b010) ? add_result[31:0] : // ADD运算
34                 (ALUop == 3'b110) ? add_result[31:0] : // SUB运算
35                 (ALUop == 3'b111) ? slt_result : // SLT运算
36                 `DATA_WIDTH'd0; // 默认值, 避免不定态
37 // ===== 第三步: 标志位计算 (纯组合逻辑) =====
38 // 1. Zero标志: Result全0时为1
39 assign Zero = (Result == `DATA_WIDTH'd0);
40 // 2. Overflow (有符号数溢出): 仅ADD/SUB/SLT时有效, 其他运算赋0
41 //   判定规则: 两个操作数符号相同, 且结果符号与操作数不同 → 溢出
42 assign Overflow = ((ALUop == 3'b010) || (ALUop == 3'b110) || (ALUop == 3'b111)) ?
43                 ((A[31] == B_processed[31]) && (A[31] != add_result[31])) :
44                 1'b0;
45 // 3. CarryOut (无符号数进位/借位): ADD/SUB都要计算, 其他运算赋0
46 //   - ADD: CarryOut = add_result[32] (进位)
47 //   - SUB: CarryOut = ~add_result[32] (借位 = 进位取反)
48 assign CarryOut = (ALUop == 3'b010) ? add_result[32] : // ADD: 进位

```

```

49         (ALUOp == 3'b110) ? ~add_result[32] : // SUB: 借位
50         1'b0;                                // 其他运算: 0
51     endmodule

```

(c) 仿真波形与结果说明本模块通过功能仿真完成全功能验证，覆盖常规数值、边界值、溢出场景等测试用例，各运算的仿真波形与说明如下：

i. AND 运算 (ALUOp=3'b000) 仿真波形如 Fig. 3 所示，实现两个操作数的按位与运算，所有测试用例结果均符合运算规则，Zero 标志在结果全 0 时正确置 1。

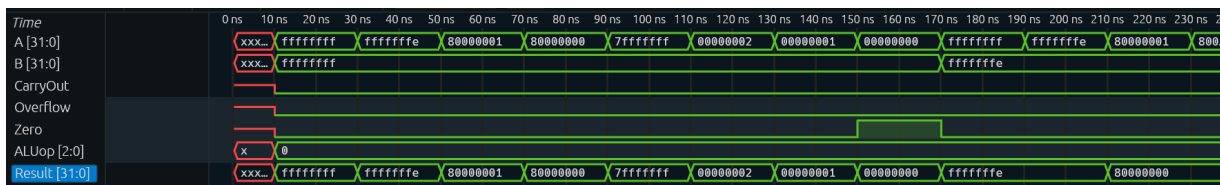


图 3: ALU AND 运算仿真波形

ii. OR 运算 (ALUOp=3'b001) 仿真波形如 Fig. 4 所示，实现两个操作数的按位或运算，所有测试用例结果均符合运算规则，标志位输出正确。

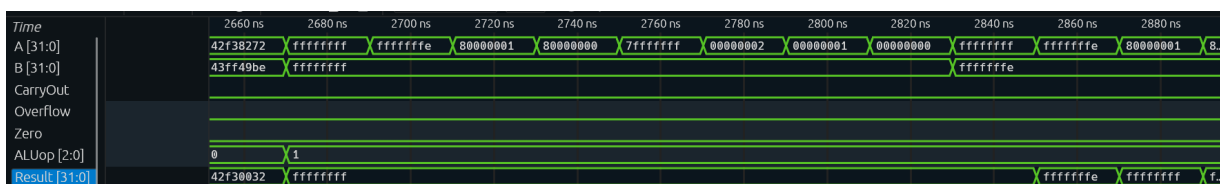


图 4: ALU OR 运算仿真波形

iii. ADD 运算 (ALUOp=3'b010) 仿真波形如 Fig. 5 所示，实现 32 位有符号数加法运算，正确输出进位标志 CarryOut 与溢出标志 Overflow，覆盖正数相加、负数相加、边界值溢出等场景，结果均符合补码加法规则。



图 5: ALU ADD 运算仿真波形

iv. SUB 运算 (ALUOp=3'b110) 仿真波形如 Fig. 6 所示，通过补码减法实现 A-B 运算，正确输出借位标志 CarryOut 与溢出标志 Overflow，覆盖正数减负数、负数减正数、同符号数相减等场景，结果均符合补码减法规则，Zero 标志在两数相等时正确置 1。

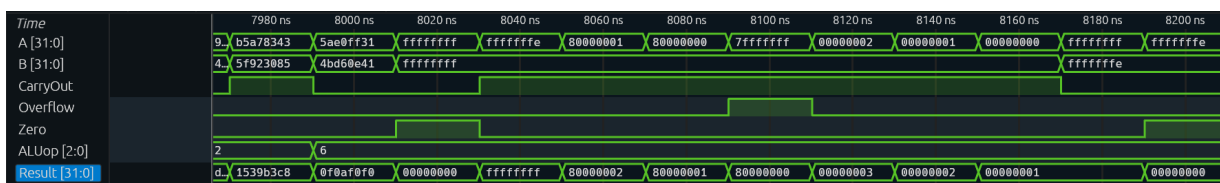


图 6: ALU SUB 运算仿真波形

- v. SLT 运算($ALUOp=3'b111$)仿真波形如图 7 所示,实现 32 位有符号数大小比较,当 $A < B$ 时输出 $32'h00000001$, 否则输出 $32'h00000000$, 覆盖同符号、异符号、边界值比较等场景,结果均符合有符号数比较规则,溢出场景下结果正确。

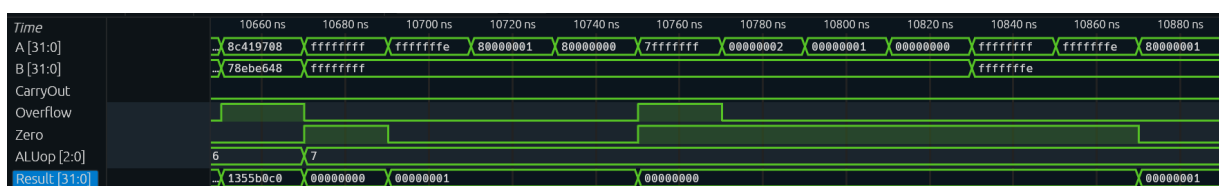


图 7: ALU SLT 运算仿真波形

3. 其他关键模块设计

本次实验仅完成寄存器堆与 ALU 两个核心模块的设计与验证,无其他关键模块,故此部分无相关内容。

二、 实验过程中遇到的问题、对问题的思考过程及解决方法

本次实验过程中遇到的核心问题、思考过程与解决方法如下:

1. 寄存器堆设计中掩码逻辑理解问题

- 问题现象:** 设计寄存器堆读端口逻辑时,最初无法理解掩码逻辑的作用,仅知道需要实现 \$zero 寄存器恒零特性,但对按位或、按位与的掩码实现原理不清晰。
- 思考过程:** 通过拆解掩码逻辑表达式,发现该逻辑本质上是通过位运算实现二选一多路选择器。当读地址的按位或结果为 0 时(地址全 0),掩码屏蔽寄存器输出,强制输出全 0;当地址不为 0 时,掩码放行寄存器输出,最终实现地址 0 恒零的特性。
- 解决方法:** 将掩码逻辑与多路选择器功能对应,明确每一位运算的作用,完成读端口逻辑设计,同时通过仿真验证了 \$zero 寄存器恒零特性符合设计要求。

2. ALU 设计中 SLT 比较逻辑冗余与溢出标志缺失问题

- 问题现象:** 最初设计 SLT 运算的 `slt_flag` 判断逻辑时,采用符号位分类讨论的方式,同时在同符号分支中加入减法结果符号位与溢出标志的异或判断,导致逻辑冗余;同时在溢出标志生成逻辑中,遗漏了 SLT 运算场景,导致异或判断逻辑失效,但分类讨论逻辑恰好让结果正确,隐藏了 bug。
- 思考过程:** 梳理有符号数比较的数学原理后,发现“减法结果符号位异或溢出标志”的判断逻辑,已经天然覆盖了符号相同和不同的所有场景,无需额外分类讨论,原有分类讨论属于冗余逻辑;同时明确 SLT 运算依赖减法操作,必须在溢出标志生成逻辑中加入 SLT 场景,否则溢出场景下的比较结果会出错。
- 解决方法:** 删除冗余的分类讨论逻辑,仅保留 '`slt_flag = add_result[31] ^ Overflow`' 核心判断;同时在 Overflow 标志的生成条件中,补充 $ALUOp=3'b111$ (SLT) 的场景,确保溢出标志在 SLT 运算中正常工作。修改后的逻辑简洁完整,无冗余,通过仿真验证了所有边界场景下的比较结果均正确。

3. SLT 结果位宽匹配的语法问题

- 问题现象:** SLT 运算的 `slt_flag` 仅为 1 位结果,而最终输出需要 32 位数据,最初采用位拼接的方式实现高位补零,但因 Verilog 语法使用不当,持续出现编译报错,无法完成综合。
- 思考过程:** 分析报错信息后,发现位拼接语法使用存在问题,而 Verilog 中对向量的分段赋值有明确的语法规则,直接对向量的不同位段分别赋值,是更直观且不易出错的实现方式。

(c) 解决方法: 放弃位拼接的实现方式, 改为分别对 `slt_result` 的第 0 位和高 31 位进行赋值, 将第 0 位赋值为 `slt_flag`, 高 31 位直接赋值为 31 位全 0。修改后的代码编译通过, 无语法错误, 通过仿真验证了 SLT 结果的位宽与数值均符合设计要求。

三、 对讲义中思考题的理解和回答

本次实验讲义未设置相关思考题, 故此部分无相关内容。

四、 实验所耗时间

在课后, 我花费了大约 2 小时完成此次实验的设计、仿真与实验报告撰写工作。